Day 16
RAII & Smart Pointers

Preston Seay && Emily Ai

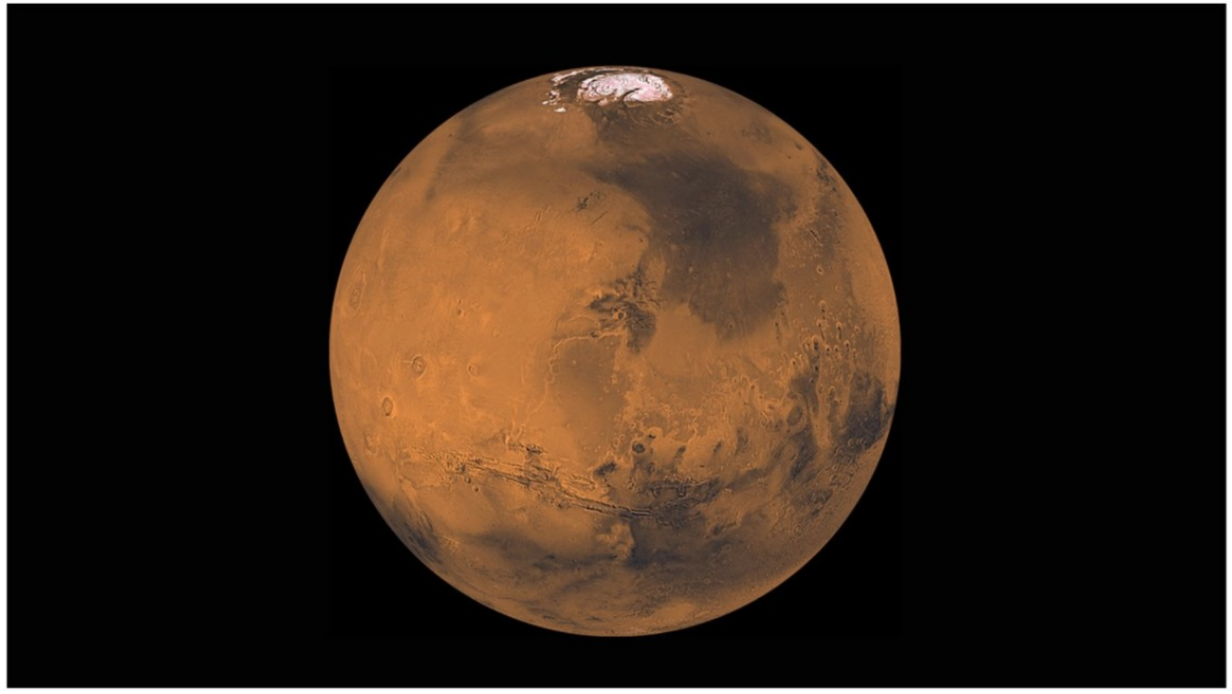
Recap - What's the problem with this?

```
1 void adjustToCorrectSpeed() {
2   int distanceToMars = getDistanceToMars();
3   int speed = getSpeed(distanceToMars);
4   if (speed < targetSpeed()) {
5     speedUp();
6   } else {
7     slowDown();
8   }
9 }
```

Recap - What's the problem with this?

Mars Climate Orbiter Team Finds Likely Cause of Loss

Sept. 30, 1999



"The peer review preliminary findings indicate that one team used English units (e.g., inches, feet and pounds) while the other used metric units for a key spacecraft operation. This information was critical to the maneuvers required to place the spacecraft in the proper Mars orbit."

https://www.jpl.nasa.gov/news/mars-climate-orbiter-team-finds-likely-cause-of-loss/

Warm-up - Right color of banana?

https://www.106l.com/live/raii-smart-pointers/warm-up?key=0of4pybgmoC1jg0NJb71g2fG08GZtHSfIHtywR6LHoc ↗

Plan

1. **RAII**
2. Smart pointers
3. Building C++ projects

Every Fruit Has a Lifetime 🍓

🛒 Buy it

```
new Fruit("banana")
```

🍌 Use it

```
banana->getName()
```

🗑️ Compost it

```
delete banana
```

Forget the last step and the fruit rots in memory: a **memory leak** 🪰

Code With Many Paths is Hard to Check

```
1  void lunch(bool hungry) {
2      Fruit* banana = new Fruit("banana");
3      if (hungry) {
4          return;          // 🪰 banana rots!
5      }
6      banana->getName();   // could throw!
7      delete banana;
8  }
```

Every early `return` skips the `delete`

So does every **exception**

More ways out = more places to forget cleanup!

All Resources Must be Released

- **Heap memory:** `new` → `delete`
- **Files:** `open()` → `close()`
- **Locks:** `lock()` → `unlock()`
- **Network sockets:** `connect()` → `close()`

RAII: Resource Acquisition Is Initialization

- **Acquire** the resource in the **constructor**
- **Release** it in the **destructor**

Q: Why does this work?

A: Destructors run on every way out of a scope.

RAII in Action: A Lunchbox 

```cpp
class Lunchbox {
public:
    Lunchbox(std::string name)
        : fruit(new Fruit(name)) {}
    ~Lunchbox() { delete fruit; }

private:
    Fruit* fruit;
};
```

Buy in the constructor, compost in the destructor

```cpp
void lunch(bool hungry) {
    Lunchbox box("banana");
    if (hungry) {
        return;  // ~Lunchbox()
    }
    // ...
}                // ~Lunchbox()
```

No `delete` in `lunch`: every path cleans up!

You've Probably Already Used RAII!

```cpp
{
    std::ifstream file("fruits.txt");
    // ...
}   // destructor closes it
```

✅ Constructor opens, destructor closes

`std::vector` and `std::string` free their own memory the same way

Think About It With a Partner!

Is this RAII? What goes wrong if the fridge is empty?

```
1  void grabSnack(std::vector<Fruit*>& fridge) {
2      fridgeLock.lock();
3      if (fridge.empty()) return;
4      fridge.grabFruit();
5      fridgeLock.unlock();
6  }
```

Not RAII: the early `return` leaves the fridge **locked forever** 🔒

Fix: `std::lock_guard` unlocks in its destructor

Activity: Pack a Lunchbox 

106l.com → RAII & Smart Pointers → slide 2

Plan

1. RAII
2. **Smart pointers**
3. Building C++ projects

Smart Pointers: RAII for Memory

A smart pointer **owns** a heap object and `deletes` it in its destructor

- `std::unique_ptr`: **one** owner 🍌
- `std::shared_ptr`: **many** owners 🍍
- `std::weak_ptr`: **no** owner, just watching 👀

All three live in `<memory>`

std::unique_ptr: only one!

```
1 auto kiwi = std::make_unique<Fruit>("kiwi");
2 kiwi->getName();                          // use it like a pointer
3
4 auto twin = kiwi;                         // ❌ compile error: no copies!
5 auto friendKiwi = std::move(kiwi); // ✅ hand it over
```

Can't copy: two owners would `delete` it twice 💥

Can move: it changes hands, still one owner

Always create them with `std::make_unique`, never `new`!

std::shared_ptr: shared ownership 🍍

```cpp
// Whatever we pass goes straight into the constructor of the "pointee."
auto pineapple = std::make_shared<Fruit>("pineapple"); // count: 1
{
    auto roommate = pineapple;          // count: 2
    auto otherRoommate = pineapple;     // count: 3
}                                       // count: 1
pineapple = nullptr;                    // count: 0, composted! 🗑️
```

Copies **share** one fruit and a **reference count**

The **last** owner to leave composts it

Uh Oh: A Fruit Salad Cycle 🍎🍊

```cpp
struct SaladFruit {
    std::shared_ptr<SaladFruit> neighbour;
};

void makeSalad() {
    auto apple = std::make_shared<SaladFruit>();
    auto orange = std::make_shared<SaladFruit>();
    apple->neighbour = orange;
    orange->neighbour = apple;
}
```

- The apple keeps the orange alive 🍊
- The orange keeps the apple alive 🍎

Neither count ever reaches 0, so both rot forever 🪰

std::weak_ptr: Just Looking 👀

```cpp
struct SaladFruit {
    std::weak_ptr<SaladFruit> neighbour;
};
```

```cpp
// Lock to get shared ptr.
auto n = apple->neighbor.lock();
if (n) {
    // The orange is still fresh!
} else {
    // Already composted!
}
```

- Watches a fruit **without** owning it
- Doesn't change the count
- `.lock()` gives a `shared_ptr`, or `nullptr` if it's gone

Activity: Human Reference Count



Stand up when you own the pineapple 🍍

Human Reference Count 🍍

🧍 Standing = you own it (`shared_ptr`) 🙋 Hand up = just watching (`weak_ptr`)

```cpp
auto me = std::make_shared<Fruit>("pineapple"); // 1
auto frontRow = me;                             // 2
std::weak_ptr<Fruit> backRow = me;              // 3
{
    auto leftSide = me;                         // 4
}                                               // 5
frontRow = nullptr;                             // 6
me = nullptr;                                   // 7
auto check = backRow.lock();                    // 8
```

1. Preston stands: count is 1
2. Front row stands
3. Back row raises hands. Count?
4. Left side stands
5. Left side sits
6. Front row sits
7. Preston sits: count 0, compost! 🗑️
8. Back row: `nullptr`, hands down

Activity: Pass the Fruit 

106l.com → RAII & Smart Pointers → slides 3, 4 & 5

Plan

1. RAII
2. Smart pointers
3. **Building C++ projects**

One File, One Command

```
g++ -std=c++20 main.cpp -o main
```

- `g++`: the compiler
- `-std=c++20`: which C++ version
- `main.cpp`: the source files
- `-o main`: name the executable

TensorFlow has **2,000+** source files... 😭

make: A Recipe for Your Project 👨‍🍳

```
1 CXX = g++
2 CXXFLAGS = -std=c++20
3 SRCS = $(wildcard *.cpp)
4
5 all:
6   $(CXX) $(CXXFLAGS) $(SRCS) -o main
7
8 clean:
9   rm -f main
```

- A **Makefile** says how to build
- Just run `make`
- Only rebuilds what **changed**

CMake: Writes the Recipe for You

```
cmake_minimum_required(VERSION 3.10)
project(orchard)
set(CMAKE_CXX_STANDARD 20)
file(GLOB SRC_FILES "*.cpp")
add_executable(main ${SRC_FILES})
```

1. `mkdir build && cd build`
2. `cmake ..` writes a Makefile
3. `make` builds it
4. `./main` runs it

CMake is a build system **generator**: one file, any computer

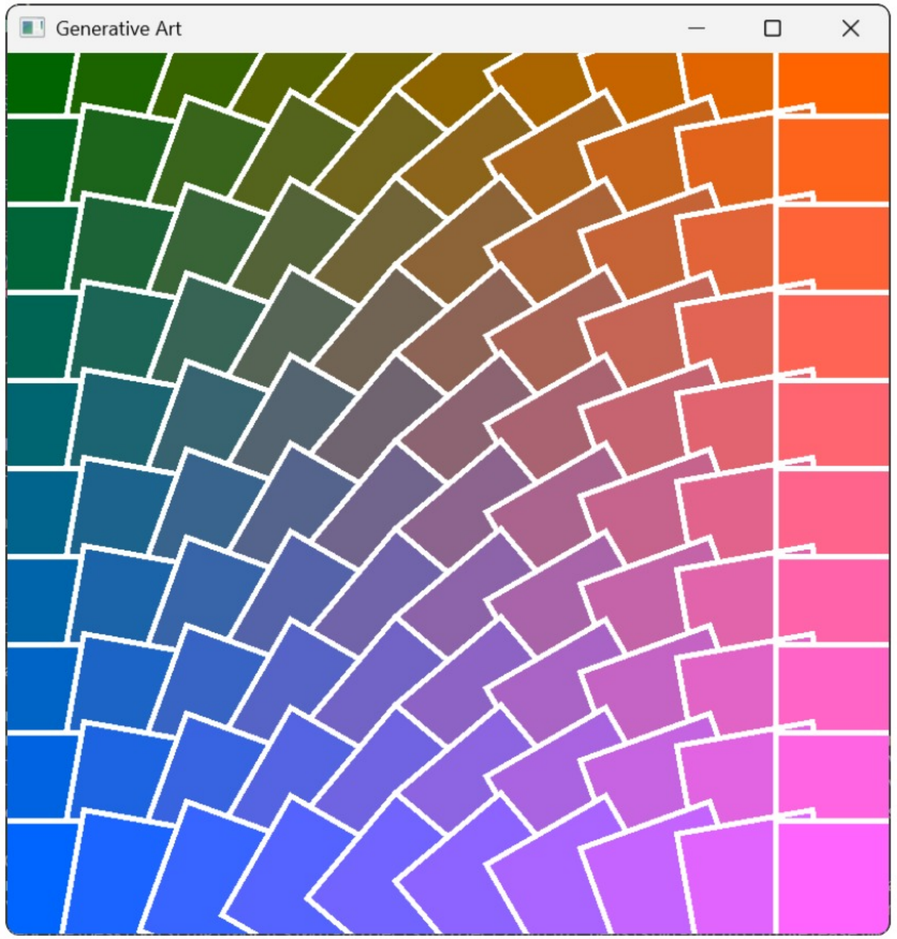
Activity: Build an Orchard 🌳

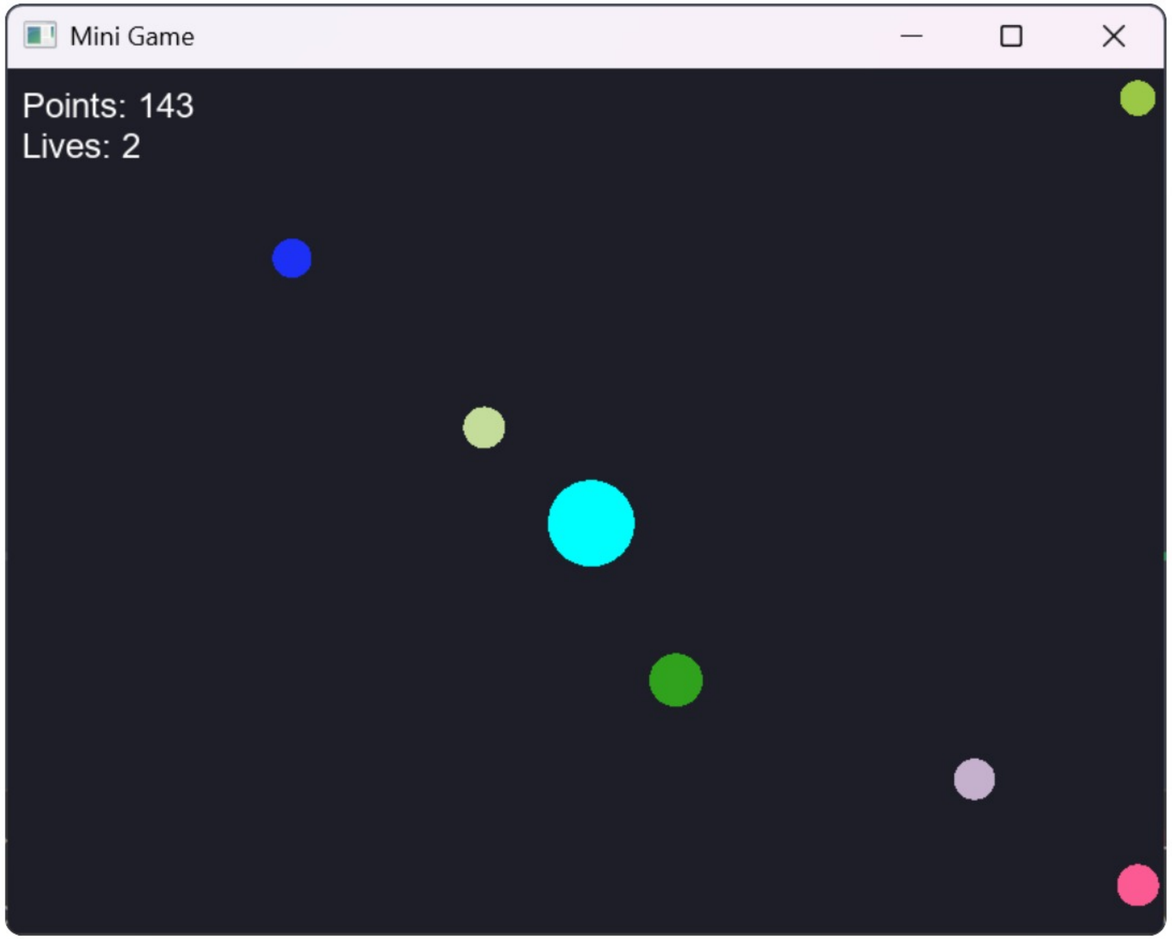
106l.com → RAII & Smart Pointers → slides 6 & 7

Recap

1. Every resource has a lifetime: acquire it, then **release** it.
2. **RAII**: the constructor acquires, the destructor releases.
3. `unique_ptr` owns alone, `shared_ptr` shares, `weak_ptr` watches.
4. Skip `new`: use `make_unique` and `make_shared`.
5. Build big projects with `make` and CMake.

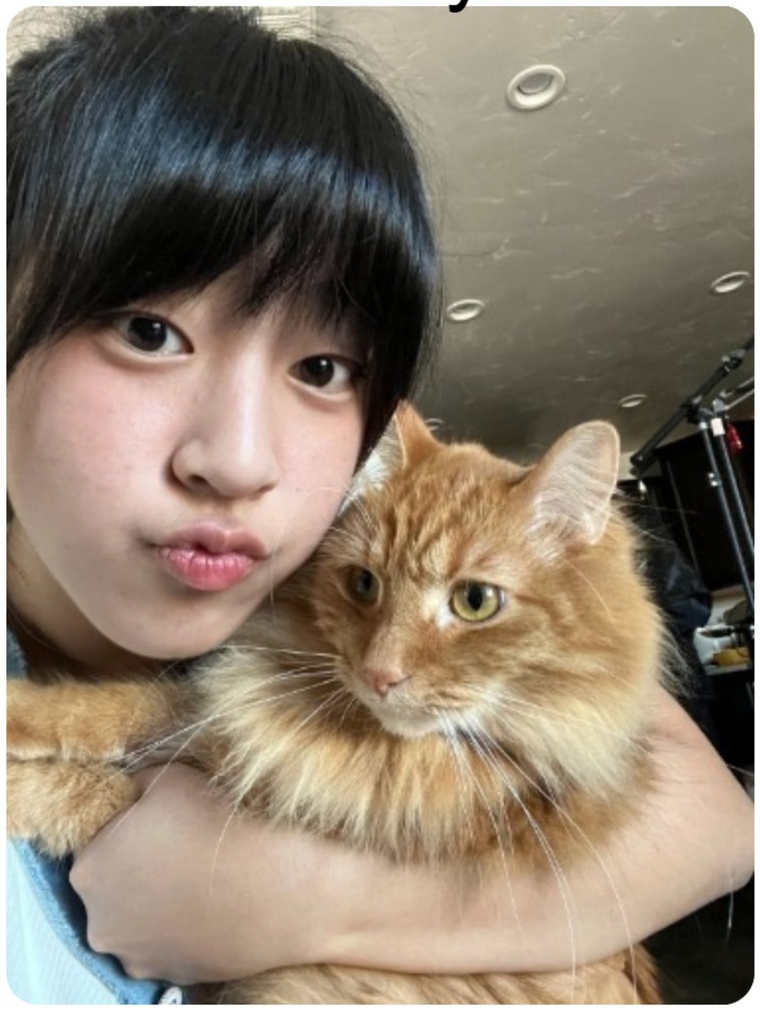
Final Project: Using a Multimedia Library





Goodbye :(

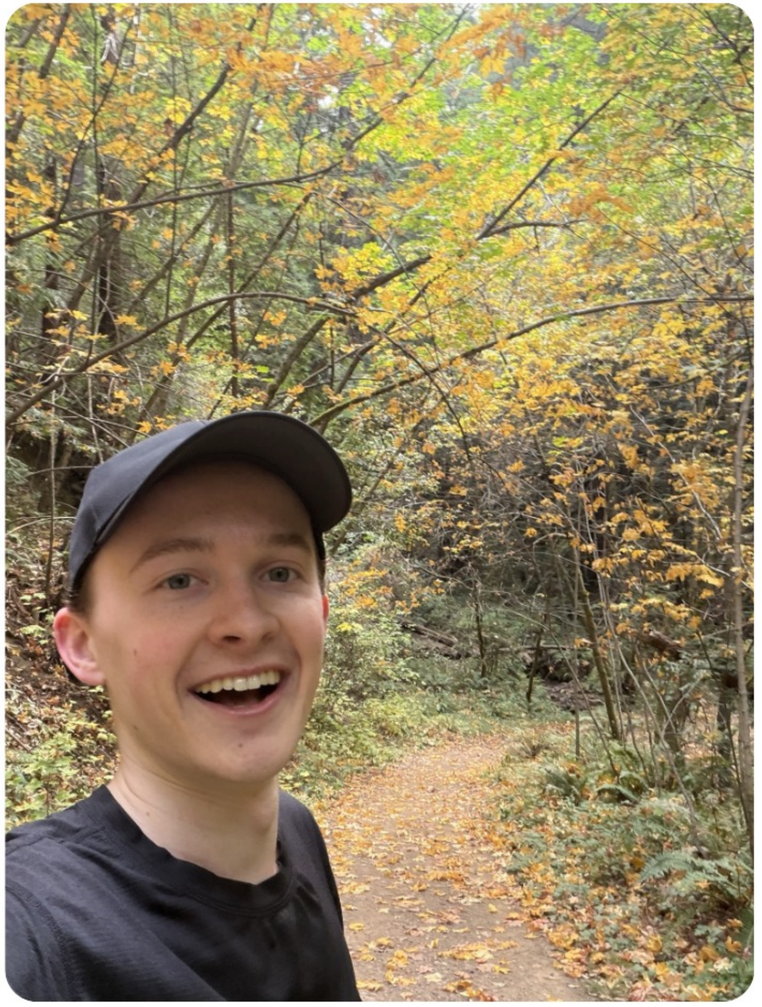
Emily



xai@stanford.edu

We have an optional lecture on Tuesday; and we'll miss you. Good luck with the rest of your quarter!

Preston



pseay@stanford.edu